

Lecture 4: CIFAR-10, CNNs, Regularisation
Class Activation Maps, Adversarial Attacks
Ising Model, Spin Glasses, Hopfield Network

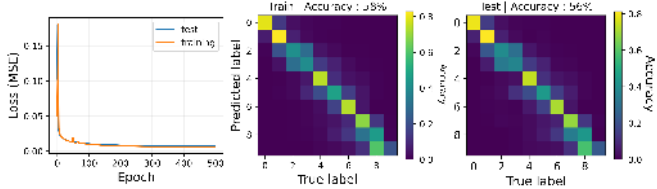
Deep Learning in Classical and Quantum Physics

Dr Marcin Płodzień

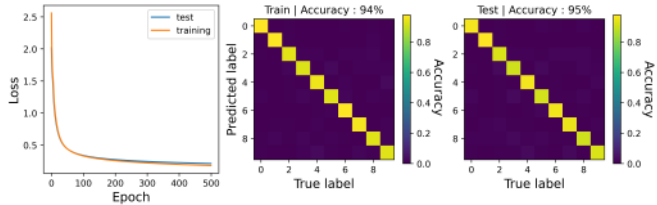
Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

Recap: MNIST (Modified National Institute of Standards and Technology) Digits

dense, 1 output, MSE (mean squared error)
train 58%, test 56%



dense, 10 outputs, cross-entropy
train 94%, test 95%



The formulation of the problem matters as much as the network.

Today: the same recipe on a harder dataset.

1

Hello World #2: CIFAR-10 and Convolutional Neural Networks (CNNs)

JAGIELLONIAN UNIVERSITY
IN KRAKOW

CIFAR-10 Dataset (Canadian Institute For Advanced Research)



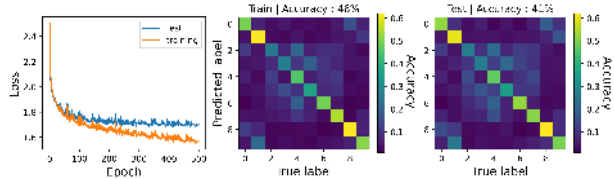
- 10 classes: aeroplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck
- 50 000 training, 10 000 test images
- 32×32 pixels in **3 colour channels** (R, G, B): each image is a $32 \times 32 \times 3$ tensor
- MNIST was $28 \times 28 \times 1$

A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images* (2009)

CIFAR-10: The Same Naive Network + Cross-Entropy

Exactly the MNIST recipe:

- **flatten** the image:
  $32 \cdot 32 \cdot 3 = 3072$ inputs
- one hidden layer, 21 sigmoid nodes
- 10 outputs, softmax, cross-entropy



$3072 \cdot 21 + 21 + 21 \cdot 10 + 10 = 64\,753$

parameters

- **Poor performance:** about 46% training, 41% test accuracy — the model *underfits*

What do we lack in our approach?

Flattening Loses Spatial Structure

Neighbouring pixels are correlated: an edge, a texture, an eye are *local* patterns of a few nearby pixels.

After flattening, the dense layer treats pixel #1 and pixel #2 as unrelated as pixel #1 and pixel #3000.

Shuffle all pixels with one fixed permutation: a dense network learns equally well — it never used the 2D layout.

What an image model should know

1. **Locality:** look at small neighbourhoods
2. **Weight sharing:** the same edge detector is useful everywhere in the image
3. **Translation:** a cat in the corner is still a cat

$\Rightarrow$ **capture spatial correlations** with *convolutions*.

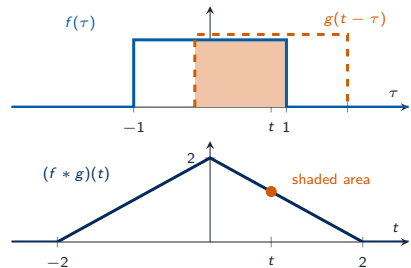
Convolution: the Continuous Definition (1D)

Two functions $f(\tau)$ (the signal) and $g(\tau)$ (the kernel):

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t - \tau)\, d\tau$$

Four steps for each output point t:

1. **flip** the kernel: $g(\tau) \to g(-\tau)$
2. **shift** it by t: $g(t - \tau)$
3. **multiply** with the signal: $f(\tau)\, g(t - \tau)$
4. **integrate** over τ: one number, $(f * g)(t)$



Example: two unit boxes on $[-1, 1]$. The overlap of the box with the shifted box has length $2 - |t|$, so $(f * g)(t) = \max(0,\, 2 - |t|)$: a **triangle**.

In physics: a measured spectrum = true spectrum $*$ instrument response; a blurred photo = sharp image $*$ point-spread function.

Convolution: the Continuous Definition (2D)

For functions of two variables, e.g. an image $f(x, y)$ and a kernel $g(x, y)$:

$$(f * g)(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(u, v)\, g(x - u,\, y - v)\, du\, dv$$

The same four steps: flip g in both directions, shift it to (x, y), multiply, integrate over the plane.

Properties (both in 1D and 2D)

- **linear:** $f * (ag_1 + bg_2) = a\, f * g_1 + b\, f * g_2$
- **commutative:** $f * g = g * f$ (substitute $\tau' = t - \tau$)
- **shift-equivariant:** shifting the input shifts the output by the same amount, $f(\cdot - a) * g = (f * g)(\cdot - a)$
- **convolution theorem:** $\mathcal{F}[f * g] = \mathcal{F}[f] \cdot \mathcal{F}[g]$ — in Fourier space a convolution is a product

Example: $g =$ a 2D Gaussian $\Rightarrow$ $f * g$ is f **blurred**: every point replaced by a weighted average of its neighbourhood.

Discrete Convolution in 1D

Signal sampled on a grid, $x[n]$; kernel with a few entries, $g[m]$. The integral becomes a sum:

$$(x * g)[n] = \sum_m g[m]\, x[n-m]$$

Worked example

$x = (1,\ 2,\ 4,\ 7,\ 7,\ 7), \quad g = (g[0], g[1], g[2]) = (1,\ 0,\ -1)$

$(x * g)[n] = x[n] - x[n-2]$ (only positions where the kernel fits fully):

$4-1=3, \quad 7-2=5, \quad 7-4=3, \quad 7-7=0 \quad \Rightarrow \quad x * g = (3,\ 5,\ 3,\ 0)$

- kernel $(1, 0, -1)$: a **difference** — large where the signal changes, 0 where it is flat
- kernel $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$: a **moving average** — $x * g = (2.33,\ 4.33,\ 6,\ 7)$, a smoothed signal

Shift-equivariance: prepend two zeros, $x' = (0, 0, 1, 2, 4, 7, 7, 7)$:
$x' * g = (1,\ 2,\ 3,\ 5,\ 3,\ 0)$ — the same pattern $(3, 5, 3, 0)$, moved by two places

Discrete Convolution in 2D

Image $x(i,j)$, kernel $G(m,n)$ of size $k \times k$:

$$\text{convolution:} \quad (G * x)(i,j) = \sum_m \sum_n G(m,n)\, x(i-m,\, j-n)$$

$$\text{cross-correlation (what CNN layers compute):} \quad S(i,j) = \sum_m \sum_n G(m,n)\, x(i+m,\, j+n)$$

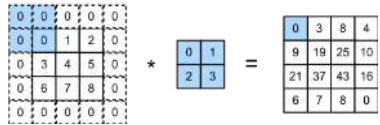
The two differ only by flipping the kernel. The kernel is *learned*, so the flip does not matter: deep-learning libraries compute the cross-correlation and still call it "convolution".

Worked example: a vertical edge

$$x = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}, \quad G = \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$S = \begin{bmatrix} 0 & 2 & 0 \\ 0 & 2 & 0 \end{bmatrix} \quad \text{(true convolution: } -2 \text{ instead of 2)}$$

e.g. middle column: patch $\begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$, $-0+1-0+1=2$: the kernel straddles the edge



2×2 kernel on a zero-padded 3×3 input: 4×4 output

With channels c: $S(i,j) = \sum_c \sum_{m,n} G(m,n,c)\, x(i+m, j+n, c)$; S is a **feature map**.

What Convolution Does

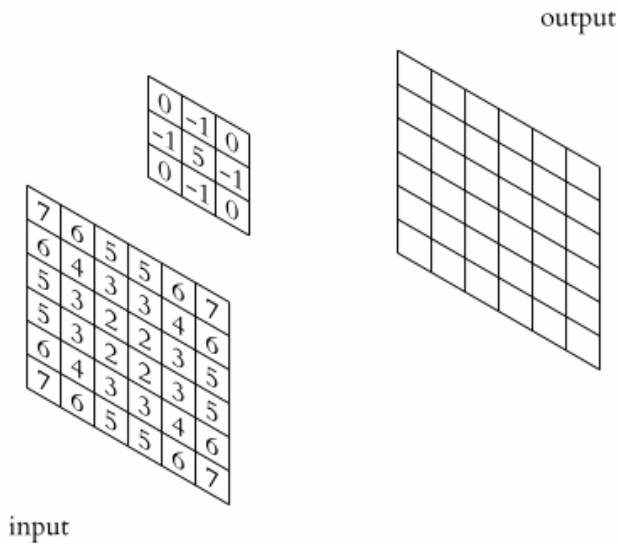
- **a local weighted sum:** each output looks only at a $k \times k$ neighbourhood
- **the same weights everywhere** (weight sharing): k^2 parameters, whatever the image size
- **a pattern detector:** $S(i,j)$ is the scalar product of the kernel with the patch at (i,j), so it is largest where the patch *looks like* the kernel
- **shift-equivariant:** move the object, the feature map moves with it

Kernel $\rightarrow$ operation	
all entries equal	blur (average)
Gaussian	smooth blur
$(-1, 1)$	derivative, edges
$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	discrete Laplacian ∇^2
centre $+$, neighbours $-$	sharpen

Physics: the Laplacian kernel is the finite-difference ∇^2 used in the heat and Schrödinger equations.

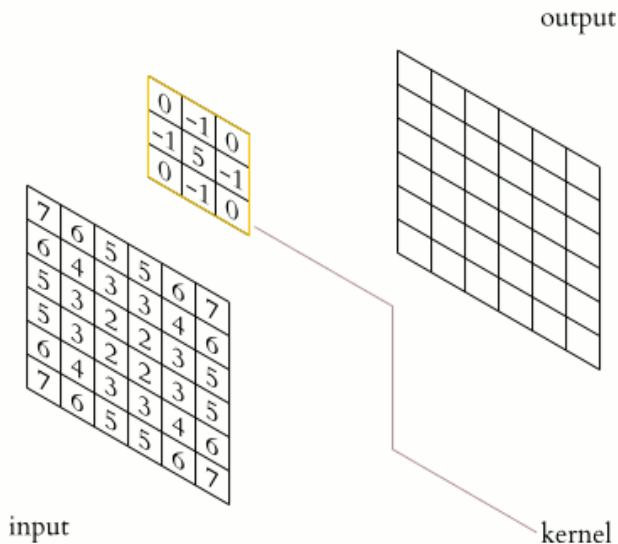
In a CNN the kernels are not designed by hand — they are learned by gradient descent.

Convolution Step by Step



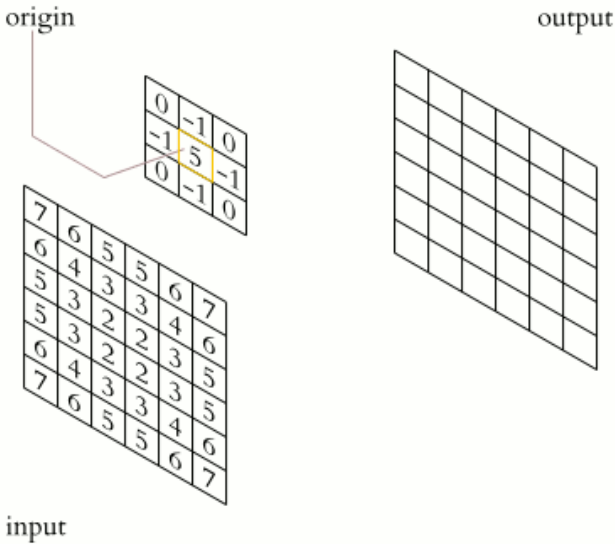
Input image, a 3×3 **kernel**, and an empty output.

Convolution Step by Step



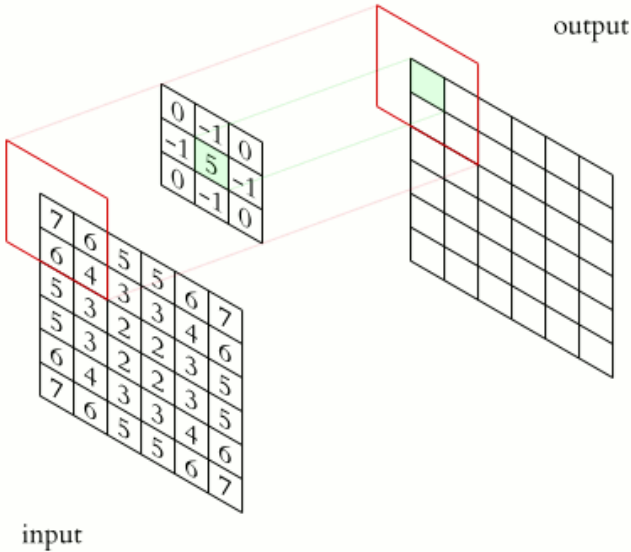
Input image, a 3×3 **kernel**, and an empty output.

Convolution Step by Step



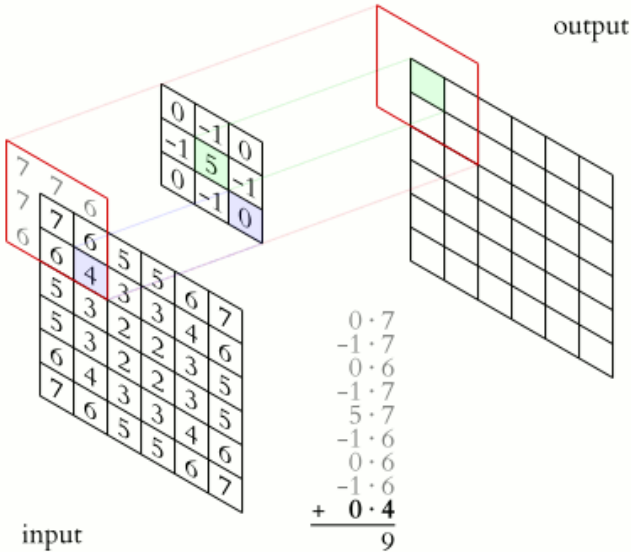
The kernel's centre ("origin") is placed on an input pixel.

Convolution Step by Step



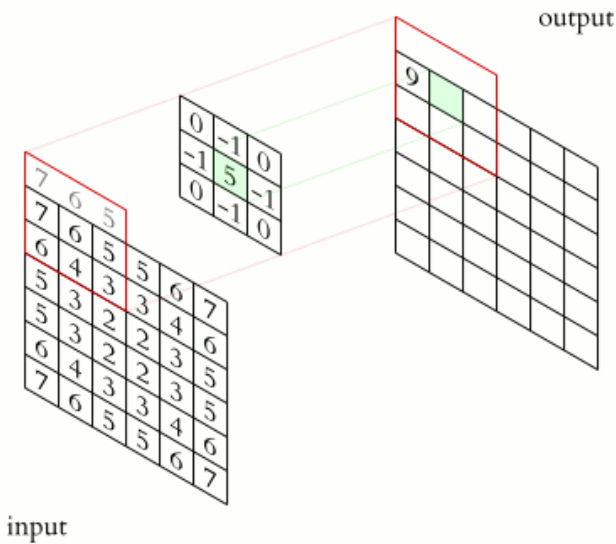
The kernel's centre ("origin") is placed on an input pixel.

Convolution Step by Step



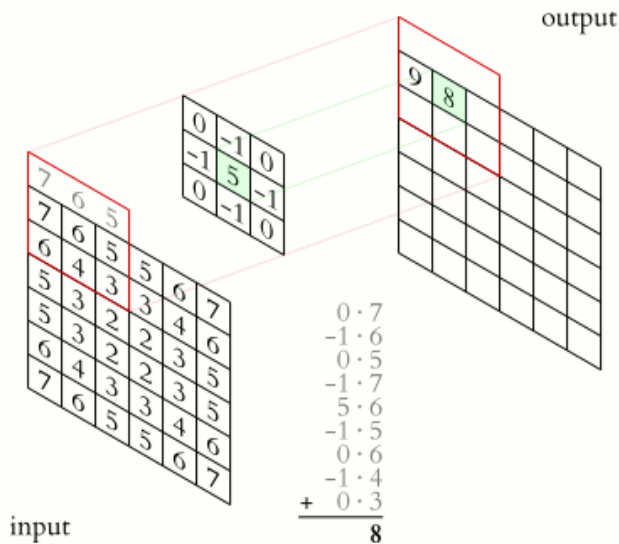
Multiply the overlapping numbers element by element and add them up: one output value.

Convolution Step by Step



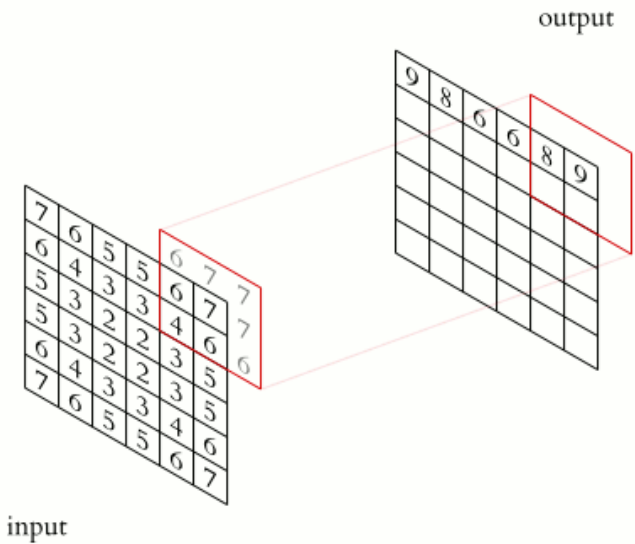
Multiply the overlapping numbers element by element and add them up: one output value.

Convolution Step by Step



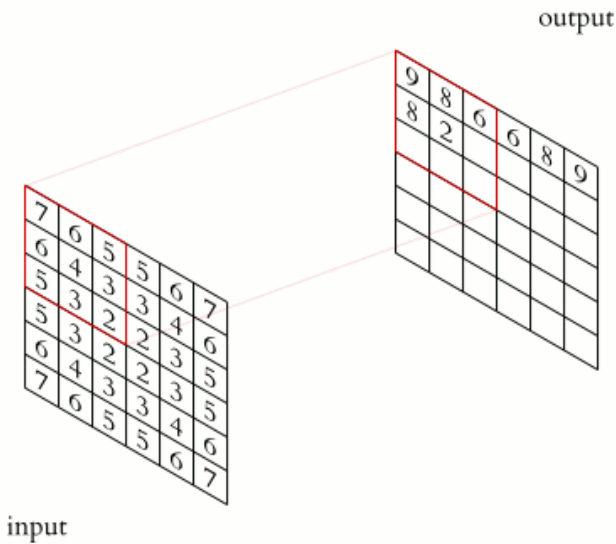
Slide by one pixel (**stride** 1) and repeat.

Convolution Step by Step



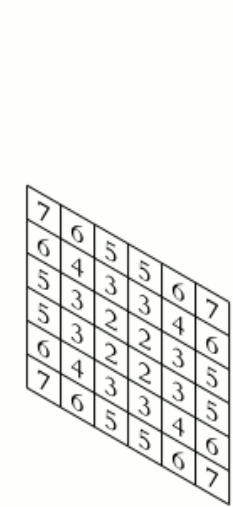
Row by row, the kernel visits every position.

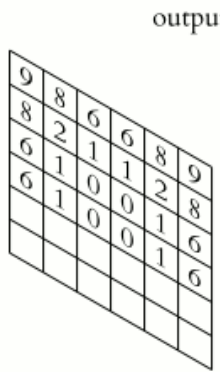
Convolution Step by Step



Row by row, the kernel visits every position.

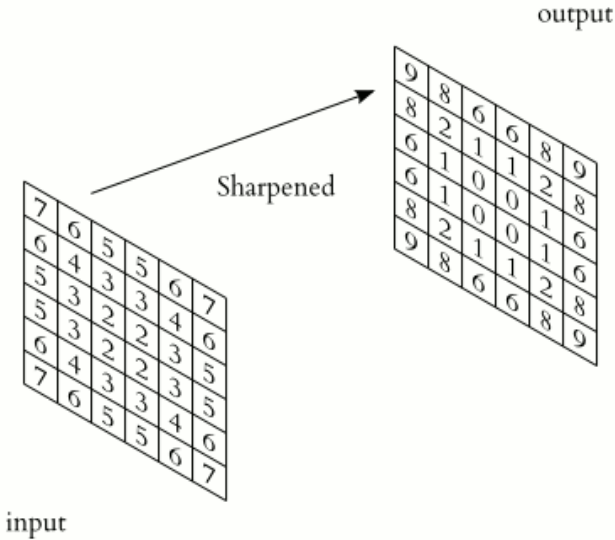
Convolution Step by Step





Row by row, the kernel visits every position.

Convolution Step by Step



This kernel, $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$, **sharpens** the image.

Kernels Detect Features: Sobel Filters

$$\vec{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \qquad \vec{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

approximate the intensity gradient: they light up where the image changes sharply — **edges**



original · vertical edges ($\vec{G}_x$) · horizontal edges ($\vec{G}_y$)

Different Kernels, Different Effects

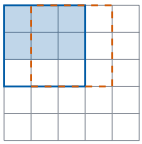
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 × 3 (approximation)	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Identity, edge detection, sharpening, blurring — all are 3×3 kernels.

Idea of a CNN: do not design the kernels — learn them.

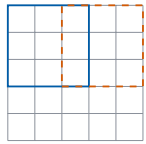
Kernel Size, Stride, Padding

stride 1



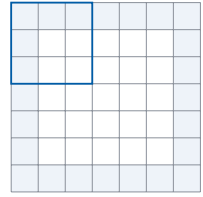
5×5 input, 3×3 kernel $\rightarrow 3 \times 3$ output

stride 2



the kernel jumps 2 pixels $\rightarrow 2 \times 2$ output

padding 1



zeros around the border $\rightarrow 5 \times 5$ output ("same")

Output size along one dimension

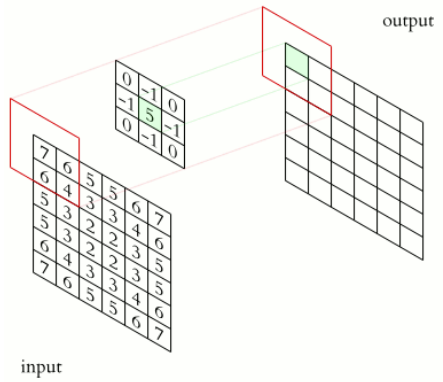
$$O = \left\lfloor \frac{W - k + 2P}{s} \right\rfloor + 1$$

W : input size, k : kernel size
P : padding, s : stride

CIFAR-10: $W = 32$, $k = 3$, $P = 1$, $s = 1 \Rightarrow O = 32$; with $s = 2$: $O = 16$

Channels and Feature Maps

- input with C_{in} channels (RGB: $C_{\text{in}} = 3$) $\rightarrow$ each kernel has shape $k \times k \times C_{\text{in}}$
- C_{out} kernels $\rightarrow$ C_{out} feature maps (output channels)
- **the kernel entries are the trainable parameters**



Parameter count, first layer on CIFAR-10

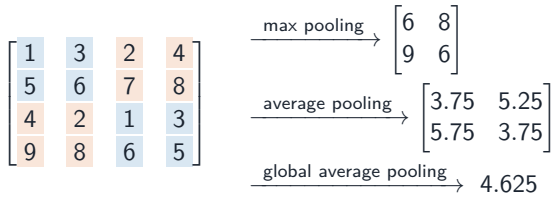
Conv2D, 32 kernels 3×3: $3 \cdot 3 \cdot 3 \cdot 32 + 32 = 896$

Dense, 21 nodes on the flattened image: $3072 \cdot 21 + 21 = 64\,533$

Fewer parameters, because one kernel is *shared* across all positions.

Pooling: Max, Average, Global Average

Window 2×2, stride 2: each 2×2 block of the feature map becomes *one* number.



top-left block: $\max(1,3,5,6) = 6$, $(1+3+5+6)/4 = 3.75$; global average: sum of all 16 numbers $= 74$, $74/16 = 4.625$

Max pooling

keeps the *strongest* response in each window: "is the feature present here?" — the usual choice between conv layers

Average pooling

keeps the *mean* response of each window: a smoothed summary

Global average pooling

the window is the *whole* map: one number per feature map; used at the end of a network

No trainable parameters in any of them. A learned alternative: a convolution with stride 2.

Downsampling: Four Reasons

1. Computational cost

one 2×2 pooling: 4 times fewer numbers to store and to process in every later layer
32 maps of 32×32: 32 768 numbers $\rightarrow$ 32 maps of 16×16: 8 192

2. A larger field of view

after pooling, a 3×3 kernel covers a 6×6 region of the map before pooling: deeper layers see larger parts of the image

3. Robustness to small shifts

a feature moved by one pixel usually stays in the same pooling window: the output does not change

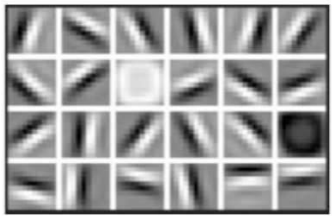
4. Fewer parameters afterwards

the dense layer after "flatten" has one weight per number that enters it: smaller maps, fewer weights, less overfitting

The price: exact positions are lost. Good for "*what* is in the image", a problem when "*where*" matters (segmentation).

CNNs Learn a Hierarchy of Features

- a CNN is a neural network that **learns the kernels** for each channel
- layers of kernels capture a **hierarchy of features**: edges $\rightarrow$ parts $\rightarrow$ objects



first layer



middle layer



deep layer

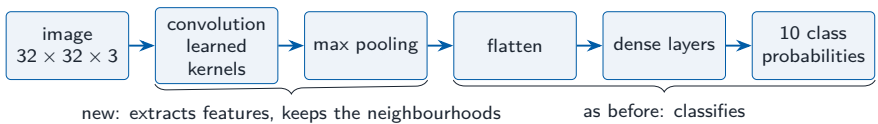
Second Attempt at CIFAR-10 Classification

First attempt: a dense network

- the image is flattened into 3072 numbers
- which pixels are neighbours is thrown away
- 46% training, 41% test accuracy

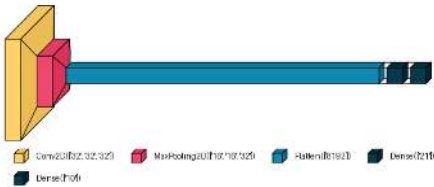
Second attempt: a convolutional network

- the image stays an image: $32 \times 32 \times 3$
- kernels look at small neighbourhoods and are *learned*
- pooling reduces the size of the feature maps

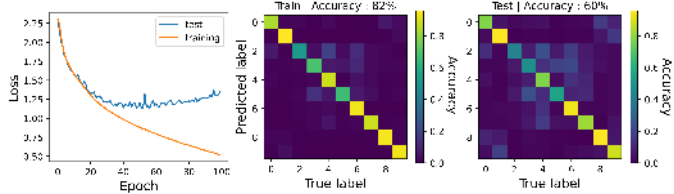


Same data, same loss (categorical cross-entropy), same training loop: **only the model changes.**

A Small CNN on CIFAR-10

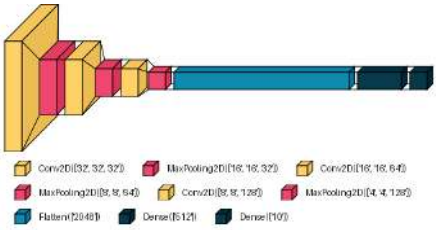


- Conv2D: 32 kernels 3×3, sigmoid, padding "same" $\rightarrow 32 \times 32 \times 32$
- MaxPooling $2 \times 2 \rightarrow 16 \times 16 \times 32$
- flatten $\rightarrow$ 8192, dense 21 (sigmoid), dense 10 (softmax)

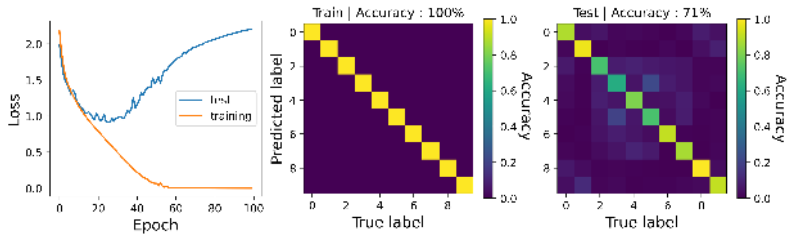


About 82% training, 60% test accuracy — better than 41%, but the test loss **rises after ∼40 epochs**: *overfitting*.

More CNN Layers: Still Overfitting



three blocks Conv2D (ReLU = rectified linear unit, $\max(0, x)$) + MaxPooling:
$32 \times 32 \times 32 \to 16 \times 16 \times 64 \to 8 \times 8 \times 128$,
then flatten, dense 512, dense 10



100% training, 71% test accuracy: **more complexity does not cure overfitting**.

2

Regularisation: Curing Overfitting



Regularisation: Three Common Techniques

Regularisation: any technique that constrains training or model capacity so that the network *generalises* to unseen data instead of memorising the training set.

1. L1 / L2 penalty	2. Dropout	3. Batch normalisation
a term in the loss that keeps the weights small	randomly switch off neurons during training	normalise the inputs of each hidden layer over a mini-batch

Also widely used: *early stopping* (stop when the validation loss starts to rise), *data augmentation* (shifted, flipped, cropped copies of the training images), *learning-rate scheduling*.

L1 / L2 Regularisation

L1 (Lasso)

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{original}} + \lambda \sum_i |\theta_i|$$

drives some weights exactly to zero: *sparse* solutions

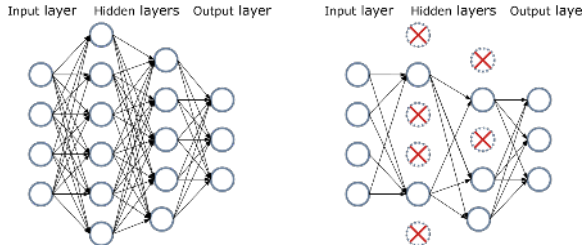
L2 (Ridge, "weight decay")

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{original}} + \lambda \sum_i \theta_i^2$$

discourages large weights: *smoother* models

- θ_i: the weights of the network; λ: the regularisation strength, a hyperparameter
- a **soft constraint**: weights are encouraged, not forced, to stay small — λ plays the role of a Lagrange multiplier
- $\lambda = 0$: no regularisation; λ too large: the network underfits

Dropout



- at every training step each neuron of the layer is switched off with probability p (the *dropout rate*, typically 0.2–0.5)
- the network cannot rely on any single neuron: it learns **redundant representations**
- each step trains a different sub-network: dropout acts as an **ensemble** of sub-networks
- at test time all neurons are active

N. Srivastava et al., *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, JMLR **15**, 1929 (2014), jmlr.org/papers/v15/srivastava14a.html

Batch Normalisation

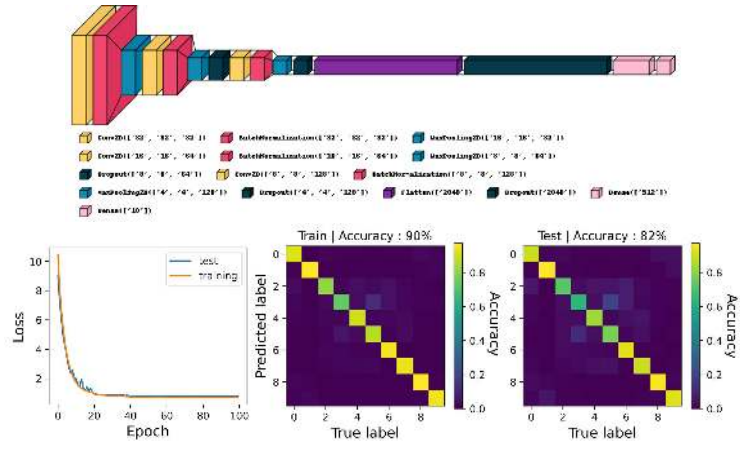
A BatchNorm (batch normalisation) layer normalises the input of a hidden layer over the mini-batch $\{\mathbf{x}_j\}_{j=1}^{N_B}$:

$$\mu_B = \frac{1}{N_B}\sum_{j=1}^{N_B}\mathbf{x}_j, \qquad \sigma_B^2 = \frac{1}{N_B}\sum_{j=1}^{N_B}(\mathbf{x}_j-\mu_B)^2 \qquad \mathbf{x}_j \leftarrow \frac{\mathbf{x}_j-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}, \qquad \mathbf{x}_j \leftarrow \gamma\,\mathbf{x}_j+\beta$$

- step 1: zero mean and unit variance within the mini-batch (ϵ: a small constant for numerical stability)
- step 2: rescale and shift with **trainable** γ and β
- makes the network less dependent on weight initialisation
- reduces sensitivity to weight changes; helps against exploding and vanishing gradients

S. Ioffe, C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, ICML 2015, arXiv:1502.03167

The Larger CNN with Regularisation



Same three Conv2D blocks, now with batch normalisation and dropout: 90% training, 82% test accuracy.

Before: 100% / 71%. **The training and test losses now stay together.**

3

Class Activation Maps: Where the CNN Looks

JAGIELLONIAN UNIVERSITY
IN KRAKÓW

Class Activation Maps

input (32 x 32)



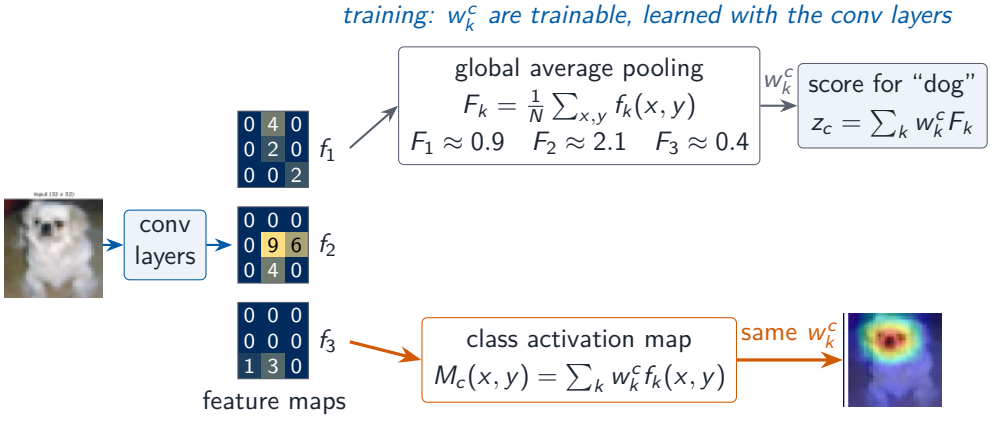
a CIFAR-10 test image; our CNN: "dog", 1.00

Which parts of the image were important for the prediction?

Can we get insight into what the CNN was looking at when it made its prediction?

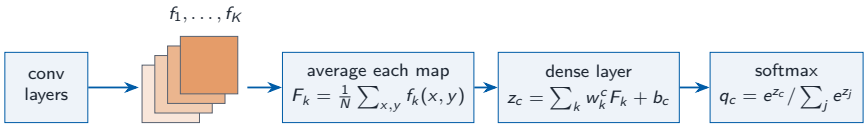
A **class activation map** (CAM) answers with a picture: a map over the image that shows *where* the evidence for a class is.

From Feature Maps to a Class Map



B. Zhou et al., *Learning Deep Features for Discriminative Localization*, CVPR 2016, arXiv:1512.04150

CAM: From the Class Score Back to a Map



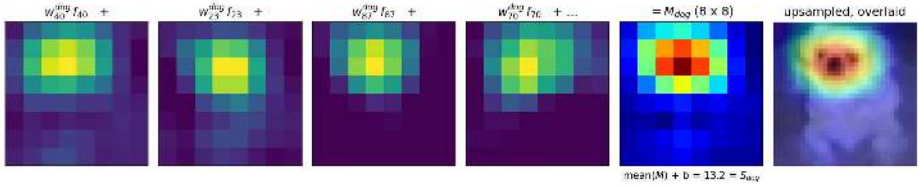
K feature maps, each $h \times w$, $N = hw$ pixels

$$z_c = \sum_k w_k^c F_k + b_c = \sum_k w_k^c \frac{1}{N} \sum_{x,y} f_k(x,y) + b_c = \frac{1}{N} \sum_{x,y} \underbrace{\sum_k w_k^c f_k(x,y)}_{M_c(x,y)} + b_c$$

Class activation map

$$M_c(x,y) = \sum_k w_k^c f_k(x,y)$$ the class score is its average over the positions

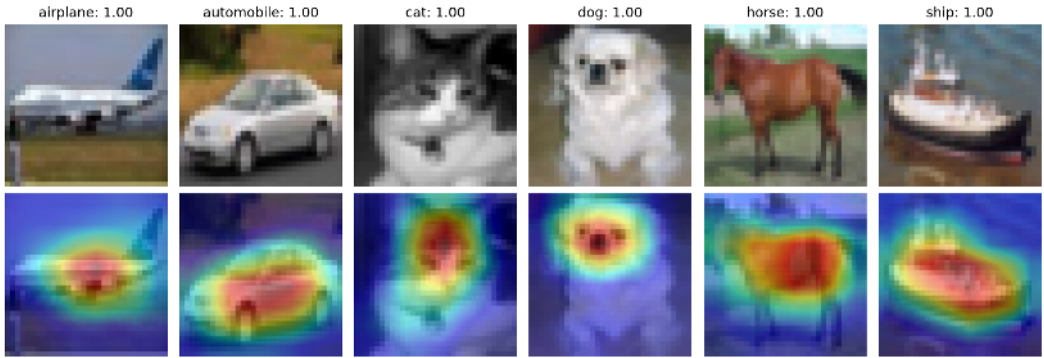
CAM on a Real Network



our CIFAR-10 CNN with GAP: 6 conv layers $\rightarrow$ 128 feature maps of 8×8 $\rightarrow$ GAP $\rightarrow$ 10 outputs, 87.6% test accuracy

top: feature maps times their weights for "dog", added to one 8×8 map; bottom: the map enlarged and laid over the image.

CAM on CIFAR-10 Test Images



Correctly classified test images (top) and their class activation maps (bottom): the evidence sits on the object, not on the background.

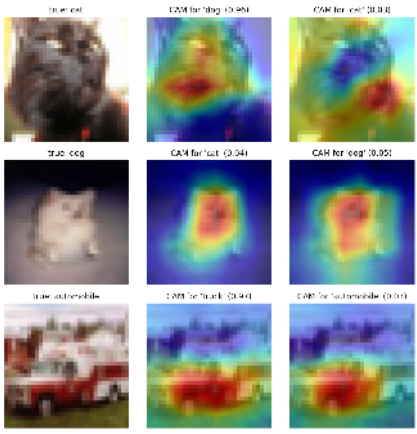
CAM for Two Classes in One Photograph



- the map of a dog class lies on the dog; the map of a cat class lies on the cat
- network: ResNet-18 trained on ImageNet (1000 classes), also ending with global average pooling
- p: softmax probability of that class. It is small: two small animals, 1000 classes to share the probability.

photograph: Socks and Buddy, U.S. National Archives, public domain, Wikimedia Commons

CAM: Looking at Mistakes



Three misclassified test images; the map for the *predicted* class (middle) and for the *true* class (right).

- **cat → "dog" (0.96):** the "dog" evidence sits on the lower part of the face; the weak "cat" evidence is elsewhere. A dark close-up, the outline of the animal is not visible.
- **dog → "cat" (0.94):** both maps cover the animal. The right place, the wrong one of two similar classes.
- **automobile → "truck" (0.97):** both maps on the body and the wheels: the right place, a neighbouring class.

Why these mistakes happen

- similar classes share most features (fur, four legs; wheels, body): their scores differ only by small weights
- 32×32 pixels: the details we would use are not in the image
- softmax must choose one of the 10 classes, and does so with high confidence even when it is wrong

CAM: Uses and Limits

Tells us

- *where* the evidence for a class is
- whether the model looks at the object or at the background (shortcut learning)
- for a mistake: compare the maps of the predicted and the true class

Does not tell us

- *why*: which property of the region mattered
- fine detail: the resolution is that of the last feature map (8×8 here), then upsampled
- anything for networks that do not end in GAP $\rightarrow$ dense

Be careful: accurate for the wrong reason

a model can reach high accuracy by using the wrong evidence, e.g. telling wolves from huskies by the *snow* in the background. M. T. Ribeiro et al., *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*, KDD 2016, arXiv:1602.04938

Grad-CAM (Gradient-weighted Class Activation Map) for Any CNN

- $f_k(x, y)$: feature maps of the last convolutional layer; z_c: score of class c before softmax
- CAM needs one weight w_k^c per map; most CNNs have several layers after the last convolution, so there is no such weight
- Grad-CAM uses instead the gradient of the class score with respect to each map, averaged over the N pixels and computed by backpropagation: $\alpha_k^c = \frac{1}{N} \sum_{x,y} \frac{\partial z_c}{\partial f_k(x,y)}$

CAM

$$M_c(x, y) = \sum_k w_k^c \, f_k(x, y)$$

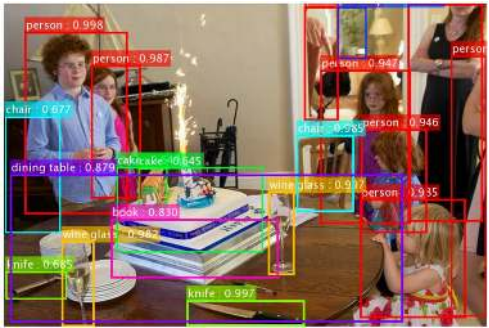
Grad-CAM

$$M_c(x, y) = \mathrm{ReLU}\Big(\sum_k \alpha_k^c \, f_k(x, y) \Big)$$

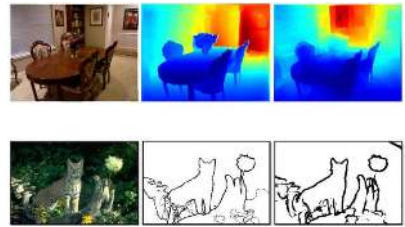
- ReLU sets negative values to zero: the map shows only the regions that *increase* z_c
- for a GAP network $z_c = \sum_k w_k^c F_k + b_c$, so $\alpha_k^c = w_k^c / N$: Grad-CAM gives the CAM map, up to the factor $1/N$ and the ReLU

R. R. Selvaraju et al., *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*, ICCV 2017, arXiv:1610.02391

Beyond Classification: Detection and Segmentation







Object detection: a box and a class probability for every object in the image

Segmentation and dense prediction: a class (or depth, or edge) for *every pixel*

All built on the same convolutional feature extractor.

Where We Are

data	model	train	test
MNIST	dense, 1 output, MSE	58%	56%
MNIST	dense, 10 outputs, cross-entropy	94%	95%
CIFAR-10	dense, 10 outputs, cross-entropy	46%	41%
CIFAR-10	small CNN	82%	60%
CIFAR-10	larger CNN	100%	71%
CIFAR-10	larger CNN with batch norm, dropout	90%	82%
CIFAR-10	CNN with GAP, batch norm, augmentation	91%	88%

first six rows: book Ch. 2; last row: our CAM network

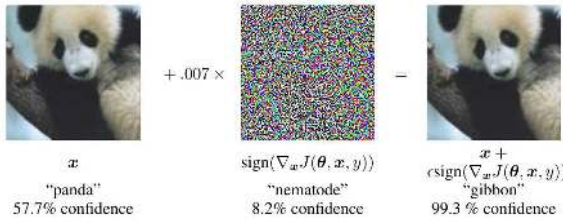
Regularisation closes the gap between training and test.

4

When Networks Fail: Adversarial Examples

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Adversarial Examples



- **left:** a photograph. The trained classifier says "panda", with 57.7% confidence — correct.
- **middle:** a pattern computed from the network. Shown alone it means nothing: the classifier's best guess is "nematode" (a roundworm), with 8.2%.
- **right:** photograph + 0.007× pattern. For us the same photograph; the classifier says "gibbon", with 99.3% confidence.

Such inputs are called **adversarial examples**.

I. J. Goodfellow, J. Shlens, C. Szegedy, *Explaining and Harnessing Adversarial Examples*, ICLR 2015, arXiv:1412.6572

Fast Gradient Sign Method

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \operatorname{sign}(\nabla_{\mathbf{x}} J(\theta, \mathbf{x}, y)), \qquad \epsilon = 0.007$$

$J(\theta, \mathbf{x}, y)$	the loss (cross-entropy) of the network with weights θ for the image $\mathbf{x}$ and its true label y
$\nabla_{\mathbf{x}} J$	the gradient with respect to the **image**: one number per pixel value — "if this pixel becomes slightly brighter, how much does the loss grow?"
$\operatorname{sign}(\cdot)$	keep only the direction: $+1$ (brighter raises the loss) or -1 (darker raises the loss)
ϵ	the size of the step, the same for every pixel

Training (what we did so far)

the image is fixed, the *weights* move *down* the loss:

$\theta \leftarrow \theta - \eta \nabla_\theta J$

Attack

the weights are fixed, the *image* moves *up* the loss:

$\mathbf{x} \leftarrow \mathbf{x} + \epsilon \operatorname{sign}(\nabla_{\mathbf{x}} J)$

Nothing is trained here. There is no second network that generates the pattern. The classifier is already trained and stays untouched; the pattern is *computed* for this one image with one forward and one backward pass — the same backpropagation as in training.

Why such a small step is enough. Each pixel value changes by only ϵ, but *all* of them change, each in its worst direction. For a score that is a weighted sum, $S = \sum_i w_i x_i$, the change is $\epsilon \sum_i |w_i|$: it grows with the number of pixel values, and an image of $224 \times 224 \times 3$ pixels has 150 528 of them.

I. J. Goodfellow, J. Shlens, C. Szegedy, *Explaining and Harnessing Adversarial Examples*, ICLR 2015, arXiv:1412.6572

One-Pixel Attack



Fig. 1: CIFAR-10



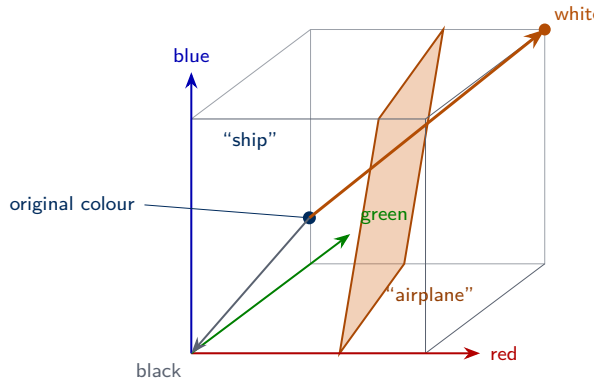
Fig. 2: ImageNet, pixel in the red circle

- black: the true class; blue: the class after changing one pixel, with the network's confidence
- Kaggle CIFAR-10 test set: 67.97% of the images can be moved to at least one other class by one pixel, with 74.03% confidence on average
- ImageNet: 16.04% of the test images, 22.91% confidence

J. Su, D. V. Vargas, K. Sakurai, *One Pixel Attack for Fooling Deep Neural Networks*, IEEE Trans. Evol. Comput. **23**, 828 (2019), doi:10.1109/TEVC.2019.2890858, arXiv:1710.08864

One-Pixel Attack: Mechanism

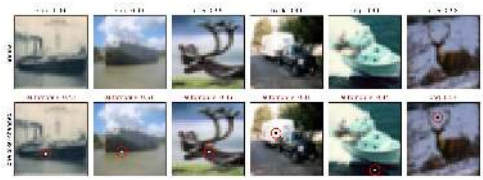
- an image is one point in a space with 3072 axes
- the network cuts this space into class regions, separated by **decision boundaries**
- one pixel = 3 of these axes: its red, green and blue value
- if a boundary is close along these 3 axes, changing the colour of that pixel moves the image into another class



the cube of all colours of *one* pixel; orange: the decision boundary. To white: the class changes. To black: it does not.

J. Su, D. V. Vargas, K. Sakurai, *One Pixel Attack for Fooling Deep Neural Networks*, IEEE Trans. Evol. Comput. **23**, 828 (2019), doi:10.1109/TEVC.2019.2890858, arXiv:1710.08864

One-Pixel Attack: Finding the Pixel



The search in the paper

- 1024 positions $\times$ 256^3 colours: too many to try all
- a candidate: five numbers (x, y, R, G, B)
- its quality: the probability of the true class — the lower, the better
- differential evolution: 400 random candidates, mix them, keep the better ones, repeat
- black box: no gradients, only output probabilities

On our CIFAR-10 network

- the CNN of the CAM section (87.6% test accuracy)
- plain random search: 1000 candidates per image, extreme colours (black, white, ...)
- ? of ? correctly classified images change class (?%); average confidence in the wrong class ?%
- top: original; bottom: one pixel changed (red circle)

J. Su, D. V. Vargas, K. Sakurai, *One Pixel Attack for Fooling Deep Neural Networks*, IEEE Trans. Evol. Comput. **23**, 828 (2019), doi:10.1109/TEVC.2019.2890858, arXiv:1710.08864

Adversarial Stop Sign



left: real graffiti on a stop sign; right: the designed black and white stickers

- a few black and white stickers on a *real* stop sign
- a road-sign classifier reads it as **"Speed Limit 45"**
- 100% of the images taken in the lab, 84.8% of the video frames taken from a moving vehicle
- the perturbation survives changes of distance and viewing angle
- for an autonomous car: a stop sign that is not seen as one

K. Eykholt et al., *Robust Physical-World Attacks on Deep Learning Visual Classification*, CVPR 2018, arXiv:1707.08945

Adversarial T-Shirts



Video frames with two people. The detector (YOLOv2, "You Only Look Once") draws a box around each person it finds; the person wearing the printed pattern gets no box. Attack success rate: 74% in the digital world, 57% with a printed T-shirt on a moving person.

K. Xu et al., *Adversarial T-shirt! Evading Person Detectors in A Physical World*, ECCV 2020, arXiv:1910.11099

Attacks on Deep Networks: Inputs and Training

attack	what the attacker does	reference
at test time: the input is manipulated		
adversarial example	adds a small, designed perturbation to one input	*Intriguing properties of neural networks*, Szegedy et al. 2014
universal perturbation	one pattern that fools the network on most images	*Universal adversarial perturbations*, Moosavi-Dezfooli et al. 2017
adversarial patch	a printed sticker placed anywhere in the scene	*Adversarial Patch*, Brown et al. 2017
at training time: the data or the training are manipulated		
data poisoning	injects crafted samples into the training set	*Poisoning Attacks against Support Vector Machines*, Biggio et al. 2012
backdoor	hides a trigger: the model is normal until the trigger appears	*BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain*, Gu et al. 2017
attack on federated learning	one participant of a joint training plants a backdoor	*How To Backdoor Federated Learning*, Bagdasaryan et al. 2018

Attacks on Deep Networks: Privacy and Language Models

attack	what the attacker does	reference
against privacy and ownership: information is extracted		
model stealing	rebuilds the model from its answers to queries	*Stealing Machine Learning Models via Prediction APIs*, Tramèr et al. 2016
membership inference	finds out whether a given record was in the training set	*Membership Inference Attacks against Machine Learning Models*, Shokri et al. 2017
model inversion	reconstructs training data from the model's outputs	*Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures*, Fredrikson et al. 2015
language models		
prompt injection	hides instructions in the text or web page the model reads	*Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*, Greshake et al. 2023

Medical records, patient images, unpublished measurements: the privacy attacks concern scientific data directly.

Limits of Deep Networks

About the networks

- high test accuracy says nothing about inputs *outside* the distribution of the training data
- the network does not use the features we use: it uses whatever separates the training classes
- confidence is not correctness: the wrong answers come with high probability

In scientific data, without any attacker

- a new microscope, a different staining, another detector or hospital: the inputs shift, the accuracy can drop
- a network can learn a shortcut: a marker, a scale bar, the background, instead of the object
- tools to check: class activation maps, test data from an *independent* source

5

Optimisation Algorithms: Under the Hood



JAGIELLONIAN UNIVERSITY
IN KRAKOW

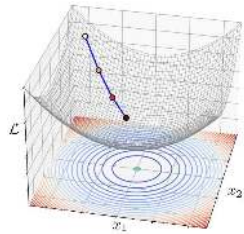
Mini-Batch Stochastic Gradient Descent (SGD)

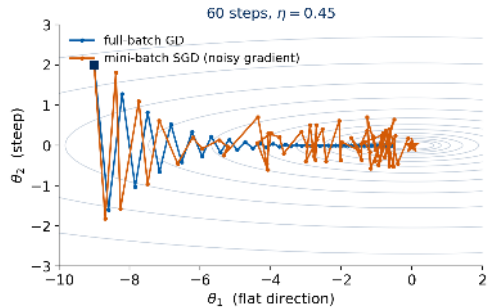
under the hood

Loss on a mini-batch $\mathcal{B}$ of size B:

$$\boldsymbol{g}_t = \nabla_{\boldsymbol{\theta}} \frac{1}{B} \sum_{i \in \mathcal{B}} \ell\big(f_{\boldsymbol{\theta}}(x_i), y_i\big)$$

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\, \boldsymbol{g}_t}$$





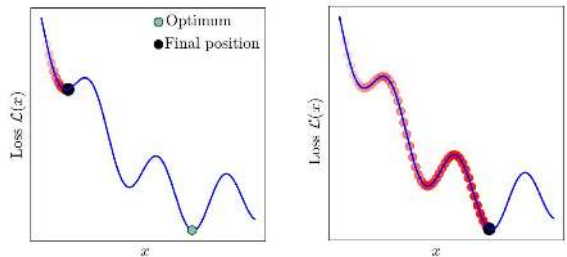
- η: learning rate — the same for every parameter
- noisy gradient: cheap per step, helps escape shallow minima
- slow in flat directions, oscillates in steep ones

SGD with Momentum

under the hood

$$\boldsymbol{v}_{t+1} = \beta\, \boldsymbol{v}_t + \eta\, \boldsymbol{g}_t, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \boldsymbol{v}_{t+1}$$

- adds a fraction of the previous update into the new one
- accumulates the history: $\boldsymbol{v}_{t+1} = \eta \sum_{k \le t} \beta^{\,t-k} \boldsymbol{g}_k$
- damps oscillations, accelerates along consistent directions
- avoids getting stuck in local minima



left: SGD stops in a local minimum; right: momentum carries it over the barrier to the optimum

A heavy ball rolling with friction $1 - \beta$.

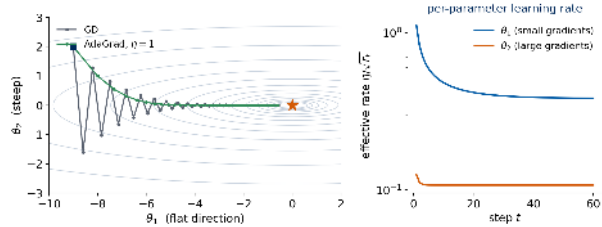
AdaGrad: Adaptive Gradient Algorithm

under the hood

$$\boldsymbol{r}_{t+1} = \boldsymbol{r}_t + \boldsymbol{g}_t \odot \boldsymbol{g}_t, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\boldsymbol{r}_{t+1}} + \epsilon} \odot \boldsymbol{g}_t$$

- adapts the learning rate **separately for each parameter**
- accumulates squared gradients
- large changes → smaller rate; small changes → relatively larger rate

Problem: $\boldsymbol{r}_t$ only grows — the rate decays towards zero.



RMSProp (Root Mean Square Propagation) / Adadelta

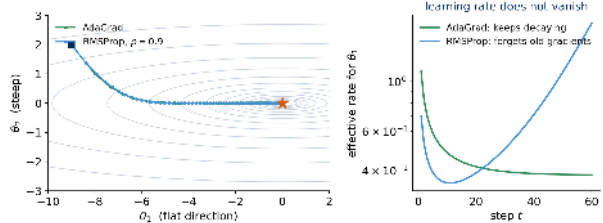


under the hood

Replace the sum by an **exponentially weighted moving average**:

$$\boldsymbol{r}_{t+1} = \rho\, \boldsymbol{r}_t + (1-\rho)\, \boldsymbol{g}_t \odot \boldsymbol{g}_t, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\boldsymbol{r}_{t+1}} + \epsilon} \odot \boldsymbol{g}_t$$

- old gradients forgotten at rate $\rho \approx 0.9$
- the learning rate no longer vanishes
- $\sqrt{\text{mean of } g^2}$ = root mean square of recent gradients — hence the name
- Adadelta ("adaptive delta): also replaces η by a running



Adam (Adaptive Moment Estimation): the Default Choice

under the hood

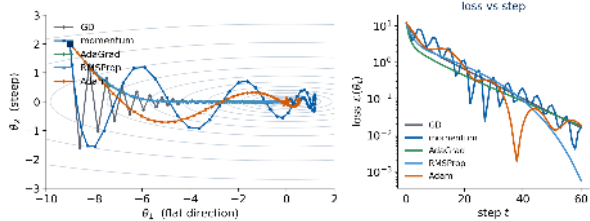
$$\boldsymbol{m}_{t+1} = \beta_1 \boldsymbol{m}_t + (1-\beta_1)\,\boldsymbol{g}_t$$

$$\boldsymbol{v}_{t+1} = \beta_2 \boldsymbol{v}_t + (1-\beta_2)\,\boldsymbol{g}_t^2$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\,\frac{\hat{\boldsymbol{m}}}{\sqrt{\hat{\boldsymbol{v}}}+\epsilon}$$

$\hat{\boldsymbol{m}} = \boldsymbol{m}_{t+1}/(1-\beta_1^{t+1})$, $\hat{\boldsymbol{v}} = \boldsymbol{v}_{t+1}/(1-\beta_2^{t+1})$

- $\boldsymbol{m}$: 1st moment (mean) of the gradient = momentum
  $\boldsymbol{v}$: 2nd moment = RMSProp scaling
- typical: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$



GD = gradient descent; toy loss $\frac{1}{2}(0.1\,\theta_1^2 + 4\,\theta_2^2)$, 60 steps; on such a simple loss the ranking depends on the tuning — Adam's strength is robustness on real networks

Adam: Kingma & Ba, arXiv:1412.6980; RMSProp: G. Hinton, Coursera lecture 6e (2012); Adadelta: M. Zeiler, arXiv:1212.5701

6

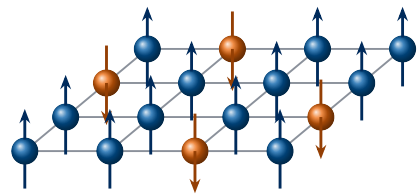
A Physics Example: the 2D Ising Model



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Classical Spins on a Lattice

- a square lattice $L \times L$; on every site i a *spin* $s_i = +1$ (up) or $s_i = -1$ (down)
- a simple model of a magnet: each spin is a tiny magnetic moment
- $2^{L \cdot L}$ possible configurations $\{s_i\}$



Energy of a configuration (Hamiltonian)

$$H(\{s_i\}) = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i$$

$\langle i, j \rangle$: pairs of nearest neighbours; $J > 0$: aligned neighbours lower the energy (ferromagnet), we set $J = 1$; h: external magnetic field, we set $h = 0$.

The Couplings Decide the Lowest-Energy State

$H = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i$: the sign of J decides which neighbours are favourable.
Lowest-energy configurations at $h = 0$:



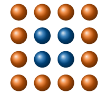
$J > 0$: ferromagnetic

neighbours *aligned*
all up or all down: 2 ground states
energy $-2J$ per spin
magnetisation $M = 1$



$J < 0$: antiferromagnetic

neighbours *opposite*
checkerboard pattern: 2 ground states
energy $-2|J|$ per spin
magnetisation $M = 0$



$J = 0$: non-interacting

spins independent
all 2^N configurations have the same energy
no order at any temperature

- square lattice: each spin has 4 neighbours, so 2 bonds per spin
- $h = 0$: flipping *all* spins leaves H unchanged — hence two equivalent ground states
- $h > 0$: spins prefer $+1$; $h < 0$: spins prefer -1; the field removes this symmetry and selects one state

blue: $s_i = +1$, orange: $s_i = -1$. From here on: $J = 1$ (ferromagnet), $h = 0$.

The State at Finite Temperature

$T = 0$: one configuration

the system sits in a configuration of minimal energy: all spins up, or all spins down

$T > 0$: a statistical ensemble

the state is *not* a single configuration, but a probability for *every* configuration (Gibbs state, $k_B = 1$)

$$p(\{s_i\}) = \frac{e^{-H(\{s_i\})/T}}{Z}, \qquad Z = \sum_{\{s_i\}} e^{-H(\{s_i\})/T} \quad \text{(partition function: the normalisation)}$$

Worked example: two spins, $H = -Js_1s_2$, four configurations, $Z = 2e^{J/T} + 2e^{-J/T} = 4\cosh(J/T)$:

configuration	**energy**	**probability**	$T \to 0$	$T \to \infty$
$\uparrow\uparrow,\ \downarrow\downarrow$	$-J$	$e^{+J/T}/Z$ each	$1/2$ each	$1/4$ each
$\uparrow\downarrow,\ \downarrow\uparrow$	$+J$	$e^{-J/T}/Z$ each	0	$1/4$ each

An observable is an average over this distribution, for example $\langle s_1 s_2 \rangle = \sum_{\{s_i\}} s_1 s_2\, p(\{s_i\}) = \tanh(J/T)$: equal to 1 at $T \to 0$ (aligned), 0 at $T \to \infty$ (independent). **Temperature sets how much weight high-energy configurations receive.**

Thermal Equilibrium: Energy against Disorder

The same rule for a lattice of $N = L^2$ spins and 2^N configurations:

$$p(\{s_i\}) = \frac{e^{-H(\{s_i\})/T}}{Z}, \qquad Z = \sum_{\{s_i\}} e^{-H(\{s_i\})/T}$$

Low temperature

low-energy configurations dominate: neighbours align, **ordered phase**

High temperature

all configurations become equally likely: spins point at random, **disordered phase**

Two observables of a configuration ($N = L^2$ spins):

$$M(\{s_i\}) = \frac{1}{N}\Big|\sum_i s_i\Big| \ \text{(magnetisation)}, \qquad E(\{s_i\}) = -\frac{1}{N}\sum_{\langle i,j\rangle} s_i s_j \ \text{(energy per spin)}$$

Thermal averages: $\langle M\rangle = \sum_{\{s_i\}} M(\{s_i\})\, p(\{s_i\})$, and the same for $\langle E\rangle$.

From the Partition Function to Observables

$Z(\beta, h) = \sum_{\{s_i\}} e^{-\beta H(\{s_i\})}$, with $\beta = 1/T$ and $H = -J \sum_{\langle i,j \rangle} s_i s_j - h\, S$, where $S = \sum_i s_i$ is the total magnetisation.

Derivative with respect to β: energy

$$\frac{\partial \ln Z}{\partial \beta} = \frac{1}{Z} \sum_{\{s_i\}} (-H)\, e^{-\beta H} = -\langle H \rangle$$

$$\frac{\partial^2 \ln Z}{\partial \beta^2} = \langle H^2 \rangle - \langle H \rangle^2$$

Derivative with respect to h: magnetisation

$$\frac{\partial \ln Z}{\partial h} = \frac{1}{Z} \sum_{\{s_i\}} \beta S\, e^{-\beta H} = \beta \langle S \rangle$$

$$\frac{\partial^2 \ln Z}{\partial h^2} = \beta^2 (\langle S^2 \rangle - \langle S \rangle^2)$$

observable	**from** $\ln Z$	**meaning**
mean energy	$\langle H \rangle = -\partial_\beta \ln Z$	first derivative: an average
mean magnetisation	$\langle S \rangle = \frac{1}{\beta}\, \partial_h \ln Z$	first derivative: an average
specific heat	$C = \dfrac{d\langle H \rangle}{dT} = \beta^2\, \partial_\beta^2 \ln Z$	second derivative: energy fluctuations
susceptibility	$\chi = \dfrac{d\langle S \rangle}{dh} = \frac{1}{\beta}\, \partial_h^2 \ln Z$	second derivative: magnetisation fluctuations

The step used twice: $\partial_x \ln Z = \frac{1}{Z} \partial_x Z$, and $\frac{1}{Z} e^{-\beta H}$ is the Boltzmann probability. Per spin: divide by N.

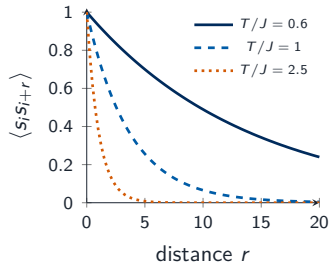
One function, $\ln Z(\beta, h)$, contains all equilibrium observables of the model.

The 1D Ising Chain

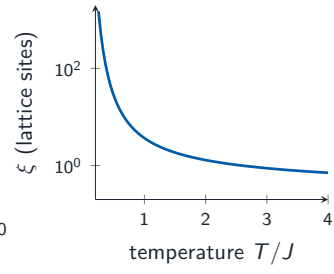
N spins on a line, $H = -J\sum_i s_i s_{i+1}$, $h = 0$. Exact solution (Ising, 1925): $Z = \left(2\cosh(J/T)\right)^N$, smooth in T for every $T > 0$.

From $\ln Z$ and the transfer matrix:

- energy per spin: $-J\tanh(J/T)$
- specific heat: $\dfrac{(J/T)^2}{\cosh^2(J/T)}$
  (smooth, no peak divergence)
- **correlations:**
  $\langle s_i s_{i+r}\rangle = \tanh^r(J/T) = e^{-r/\xi}$
- **correlation length:**
  $\xi = \dfrac{-1}{\ln\tanh(J/T)}$



exponential decay at every T



$\xi \approx \frac{1}{2}e^{2J/T} \to \infty$ only as $T \to 0$

ξ is finite at every $T > 0$:
no phase transition.

E. Ising, *Beitrag zur Theorie des Ferromagnetismus*, Z. Phys. **31**, 253 (1925), doi:10.1007/BF02980577

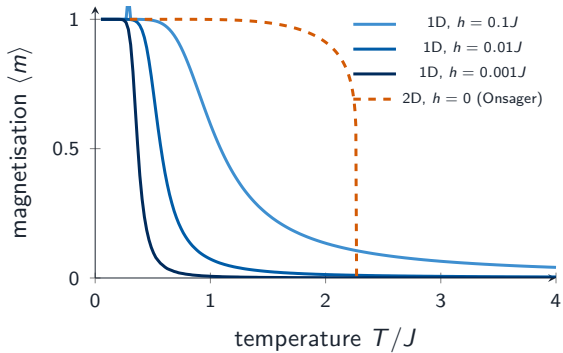
1D: Magnetisation as a Function of Temperature

Switch on a small field h to pick a direction:

$$\langle m \rangle = \frac{\sinh(h/T)}{\sqrt{\sinh^2(h/T) + e^{-4J/T}}}$$

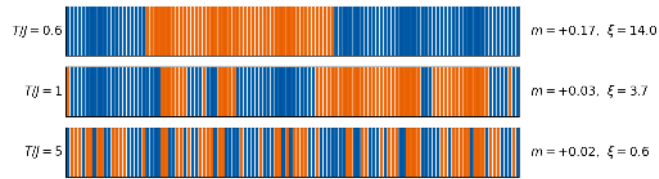
- in a field, spins align at low T
- make the field weaker: the curve shifts to $T \to 0$
- $h \to 0$: $\langle m \rangle = 0$ at *every* $T > 0$

Contrast with 2D (Onsager): $\langle m \rangle \neq 0$ below $T_c \approx 2.269\,J$ even at $h = 0$ — **spontaneous magnetisation**.



Does $\langle m \rangle = 0$ Mean the Spins Are Random?

No. Exact samples of a 1D chain ($N = 120$, $h = 0$), blue $= +1$, orange $= -1$:

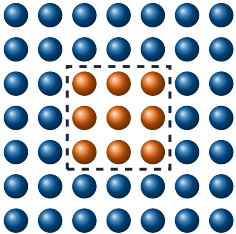


- all three have $m \approx 0$, but only the high-T chain is random
- low T: **long ordered domains**, up and down in equal measure — locally ordered, globally $m \approx 0$
- $\langle m \rangle$ is an average; the **correlation length** ξ measures how ordered the spins are

Why domains always appear in 1D: energy vs entropy

The system minimises the free energy $F = E - TS$. One domain wall costs $\Delta E = 2J$ (one broken bond), but it can sit at any of $N - 1$ places: $\Delta S = \ln(N - 1)$. So $\Delta F = 2J - T \ln(N - 1) < 0$ for a long chain at *any* $T > 0$: walls proliferate until they are $\sim \xi$ apart. In 2D a wall is a line — its cost grows with its length (next slide).

2D: Domain Walls



a flipped island; its boundary is a domain wall of length ℓ (here $\ell = 12$)

- energy cost grows with the wall: $\Delta E = 2J\ell$
- number of wall shapes of length ℓ: at most about 3^ℓ (three directions at every step), so $\Delta S \approx \ell \ln 3$
- $\Delta F \approx \ell\,(2J - T \ln 3)$: positive at low temperature, large islands are suppressed
- **order survives** below a temperature of the order of $2J/\ln 3 \approx 1.8\,J$ — a rough estimate; the exact value is $T_c \approx 2.269\,J$

Dimension decides

1D: wall energy is constant, entropy wins at every $T > 0$. 2D: wall energy grows with its length, order wins at low T.

R. Peierls, *On Ising's model of ferromagnetism*, Proc. Camb. Phil. Soc. **32**, 477 (1936), doi:10.1017/S0305004100019174

2D: Onsager's Solution of the Partition Function

Every observable is a derivative of $\ln Z$, but $Z = \sum_{\{s_i\}} e^{-\beta H(\{s_i\})}$ is a sum over 2^N configurations. **Computing $\ln Z$ in closed form solves the model.**

Onsager (1944): $\ln Z$ computed exactly for the 2D square lattice, $h = 0$, $N \to \infty$

$$\frac{1}{N}\ln Z = \ln\bigl(2\cosh(2\beta J)\bigr) + \frac{1}{2\pi}\int_0^{\pi} d\phi\, \ln\Bigl[\tfrac{1}{2}\Bigl(1+\sqrt{1-\kappa^2\sin^2\phi}\Bigr)\Bigr], \qquad \kappa = \frac{2\sinh(2\beta J)}{\cosh^2(2\beta J)}$$

- $\kappa \le 1$ for every temperature, and $\kappa = 1$ exactly when $\sinh(2\beta J) = 1$: there the square root vanishes at $\phi = \pi/2$ and $\ln Z$ is **not analytic**
- $\sinh(2J/T_c) = 1 \;\Rightarrow\; T_c = \dfrac{2J}{\ln(1+\sqrt{2})} \approx 2.269\, J$
- at T_c the specific heat C diverges logarithmically: a second-order (continuous) phase transition
- the spontaneous magnetisation $\langle M\rangle = [1 - \sinh^{-4}(2J/T)]^{1/8}$ was announced by Onsager in 1949; the derivation was published by C. N. Yang in 1952

L. Onsager, *Crystal Statistics. I. A Two-Dimensional Model with an Order-Disorder Transition*, Phys. Rev. **65**, 117 (1944), doi:10.1103/PhysRev.65.117; C. N. Yang, *The Spontaneous Magnetization of a Two-Dimensional Ising Model*, Phys. Rev. **85**, 808 (1952), doi:10.1103/PhysRev.85.808

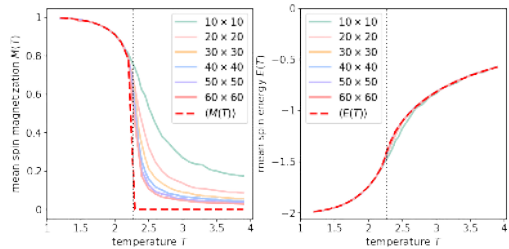
A Phase Transition at a Critical Temperature

- **1D** (Ising, 1925): no phase transition at any $T > 0$
- **2D** (Onsager, 1944, exact solution at $h = 0$): a phase transition at

$$T_c = \frac{2}{\ln(1+\sqrt{2})} \approx 2.269$$

- $T < T_c$: spontaneous magnetisation, without any external field

$$\langle M(T) \rangle = \left[1 - \sinh^{-4}\left(\frac{2}{T}\right)\right]^{1/8}$$



solid: Monte Carlo averages for $L \times L$ lattices;
dashed red: Onsager's solution ($L \to \infty$); dotted: T_c

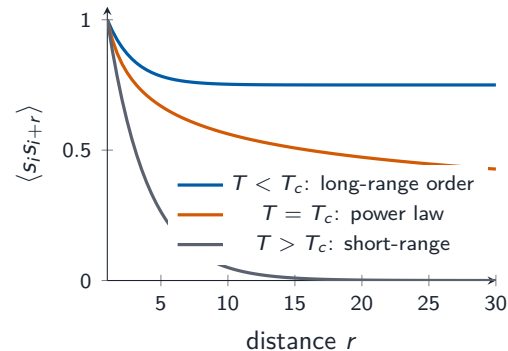
- $T > T_c$: $\langle M \rangle = 0$

M is the **order parameter**: a quantity that tells the two phases apart. Near T_c: $M \sim (T_c - T)^{1/8}$, and fluctuations of observables become large.

Criticality: Three Kinds of Correlations

Correlation of two spins at distance r: $\langle s_i s_{i+r} \rangle$. **Correlation length $\xi(T)$:** the typical cluster size.

- $T > T_c$: **short-range correlations** $\langle s_i s_{i+r} \rangle \sim e^{-r/\xi} \to 0$: exponential decay, distant spins are independent
- $T < T_c$: **long-range order** $\langle s_i s_{i+r} \rangle \to M^2 > 0$: spins stay correlated at any distance
- $T = T_c$: **critical (power-law) correlations** $\langle s_i s_{i+r} \rangle \sim r^{-1/4}$: no characteristic scale; $\xi \propto |T - T_c|^{-1}$ diverges



schematic curves (not simulation data)

Criticality: Fluctuations of Observables

The second derivatives of $\ln Z$ are *variances*: how much an observable changes from one configuration to the next.

$$\langle H^2 \rangle - \langle H \rangle^2 = T^2\, C, \qquad \langle S^2 \rangle - \langle S \rangle^2 = T\, \chi$$

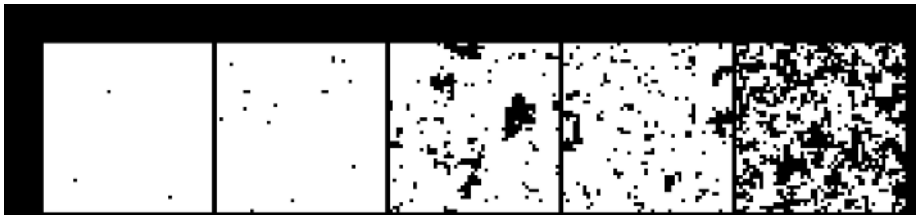
- far from T_c: small fluctuations; every sampled configuration looks like the others
- at T_c the response functions diverge: $\chi \sim |T - T_c|^{-7/4}$, $C \sim -\ln|T - T_c|$
- so the **fluctuations of magnetisation and energy are largest at T_c**: configurations at the *same* temperature differ strongly from each other

Consequence for learning

near T_c a single configuration does not reveal its phase reliably

Sampling Spin Configurations

Goal: configurations drawn from $p(\{s_i\}) \propto e^{-H/T}$ at a chosen temperature T, without summing over 2^N configurations.



sampled configurations, 50×50 lattice, at $T = 1.2,\ 1.9,\ 2.2,\ 2.4,\ 3.2$ (black and white: the two spin values)

Low T: spins aligned. High T: a random pattern. Each image is one sample from the Boltzmann distribution.

The Metropolis Algorithm

Metropolis Monte Carlo for the 2D Ising model

1. initialise an $L \times L$ lattice with random spins $s_{i,j} = \pm 1$; set the temperature T
2. pick a site (i, j) and compute the energy change of flipping its spin:

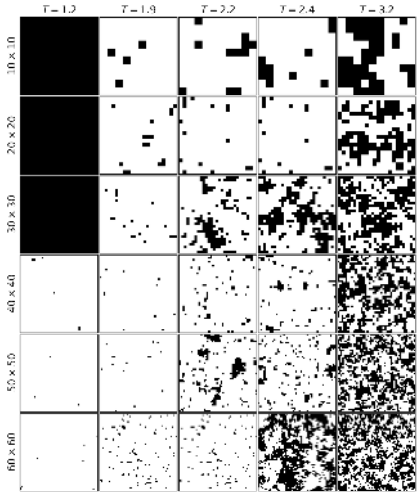
$$\Delta E = 2\, s_{i,j} \left(s_{i+1,j} + s_{i-1,j} + s_{i,j+1} + s_{i,j-1} \right)$$

3. if $\Delta E \leq 0$: flip the spin, $s_{i,j} \leftarrow -s_{i,j}$
4. if $\Delta E > 0$: draw a random number $r \in [0, 1)$ and flip the spin if $r < e^{-\Delta E / T}$
5. repeat for all sites (one Monte Carlo step); repeat many steps
6. after equilibration: store configurations and measure M, E

Only the four nearest neighbours enter ΔE. Energetically unfavourable flips are accepted more often at high T.

N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, E. Teller, *Equation of State Calculations by Fast Computing Machines*, J. Chem. Phys. **21**, 1087 (1953), doi:10.1063/1.1699114

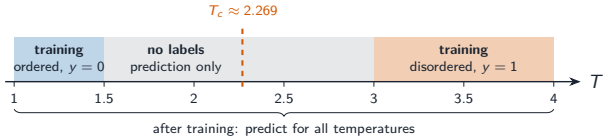
The Dataset



- rows: lattice size $L = 10, \ldots, 60$; columns: temperature T
- each configuration is a black-and-white $L \times L$ image
- low T: almost all spins aligned; high T: random pattern
- $T = 2.2$ and $T = 2.4$ look alike, but lie on **opposite sides** of $T_c \approx 2.269$

Can a network find T_c from the images alone, without being told about magnetisation?

Supervised Learning Far from the Transition

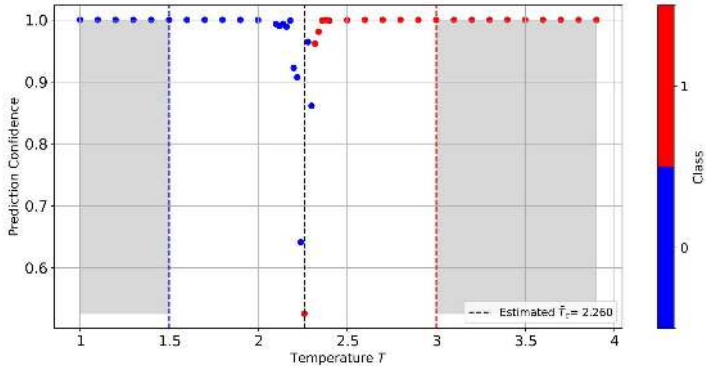


1. **Data:** X = spin configuration, y = phase label
2. **Train only where the answer is clear:**
   $T < T_{\text{min}} = 1.5$: ordered, $y = 0$; $T > T_{\text{max}} = 3$: disordered, $y = 1$
   configurations with $T_{\text{min}} \leq T \leq T_{\text{max}}$ are *excluded* from training
3. **Model:** a small CNN — two blocks of Conv2D (32 kernels 3×3) + MaxPooling, dense 64 (ReLU), 2 outputs (softmax); categorical cross-entropy, Adam optimiser
4. **Predict** on the whole temperature range, including the region the network has never seen
5. **Plot the prediction confidence** against T

Prediction confidence

the softmax probability of the predicted class: for an output $q(x) = [0.1, 0.9]$ the predicted class is 1 and the confidence is 0.9

The Confidence Drops at the Critical Temperature



$L = 50$; shaded: the training regions. Lowest confidence at $\bar{T}_c = 2.26$; exact value $T_c \approx 2.269$. Near T_c the magnetisation vanishes and fluctuations are large: the two phases are hardest to tell apart.

Interpretation of the Result

What we gained

- an estimate of T_c from raw configurations, with no formula for the order parameter
- for most physical systems the order parameter is *not known*: the network signals that *something happens* near a given temperature

Limits

- finite lattice: the transition is smoothed, the dip is broadened; larger L gives a sharper estimate
- the labels and the two training windows came from us
- the network does not tell us *why* there is a transition

Did we discover something?